Documento de Engenharia de Software

Feature Request & Product Roadmap Management System

Diagramas de Estados — Pedido e User Story

Ano Letivo 2026/27

1 Objetivo

Os diagramas de estados mostram o ciclo de vida dos dois objetos centrais do sistema: o **Pedido** (a necessidade do cliente) e a **User Story** (o trabalho de engenharia). Para cada estado indicam que acontecimentos o fazem mudar, que condições são exigidas e que ações são executadas.

2 Notação

- O círculo preto é o estado inicial e o círculo com contorno é o estado final.
- Cada seta é uma transição, com o formato `acontecimento [condição] / ação`.
- `entry /` indica uma ação executada ao entrar no estado; `do /` uma atividade que decorre enquanto o objeto está nesse estado.

3 Diagrama 1 — Pedido

O pedido é criado em **Submetido** quando o cliente o submete, guardando a data de submissão. Passa a **Em Triagem** quando o Worker conclui a triagem, com ou sem sugestões da IA. Na triagem, o Gestor de Produto tem três saídas:

- **Aprovado**, através de `aprovar()`, apenas se o esforço e o impacto estiverem definidos;
- **Rejeitado**, através de `rejeitar(justificacao)`, que é um estado final;
- fim do ciclo de vida quando o pedido é **fundido com o original** (`fundirCom()`), passando os votos e anexos para esse pedido.

Um pedido aprovado passa a **Planeado** quando é associado a uma Release, a **Em Desenvolvimento** quando a primeira User Story ligada entra em Em Progresso, e a **Lançado** quando a Release é publicada, guardando a data de lançamento. Estas três transições enviam um e-mail a quem submeteu ou votou no pedido (US 2.3).

4 Diagrama 2 — User Story

A User Story nasce em **A Fazer** através de `gerarUserStory()`. Só passa a **Em Progresso** com `iniciar()` se cumprir a Definition of Ready (tem critérios de aceitação, US 3.2) e estiver num Sprint; ao entrar, guarda a data de início e passa a verificar as dependências (US 3.3). Quando é aberto um pull request, passa a **Em Revisão**; pode voltar a Em Progresso se forem pedidas alterações, ou passar a **Concluído** quando o pull request é aprovado, guardando a data de conclusão.

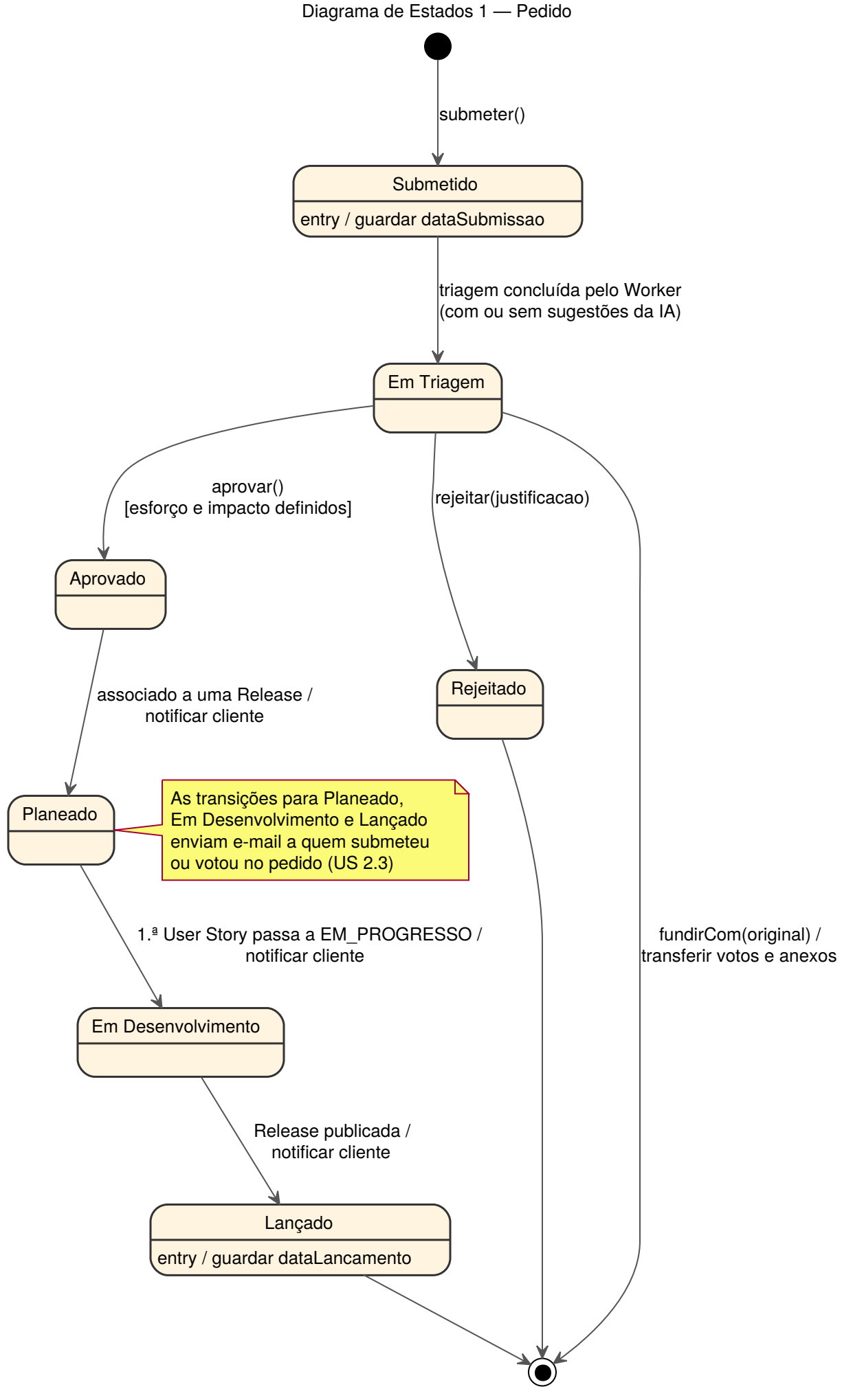
5 Decisões e regras

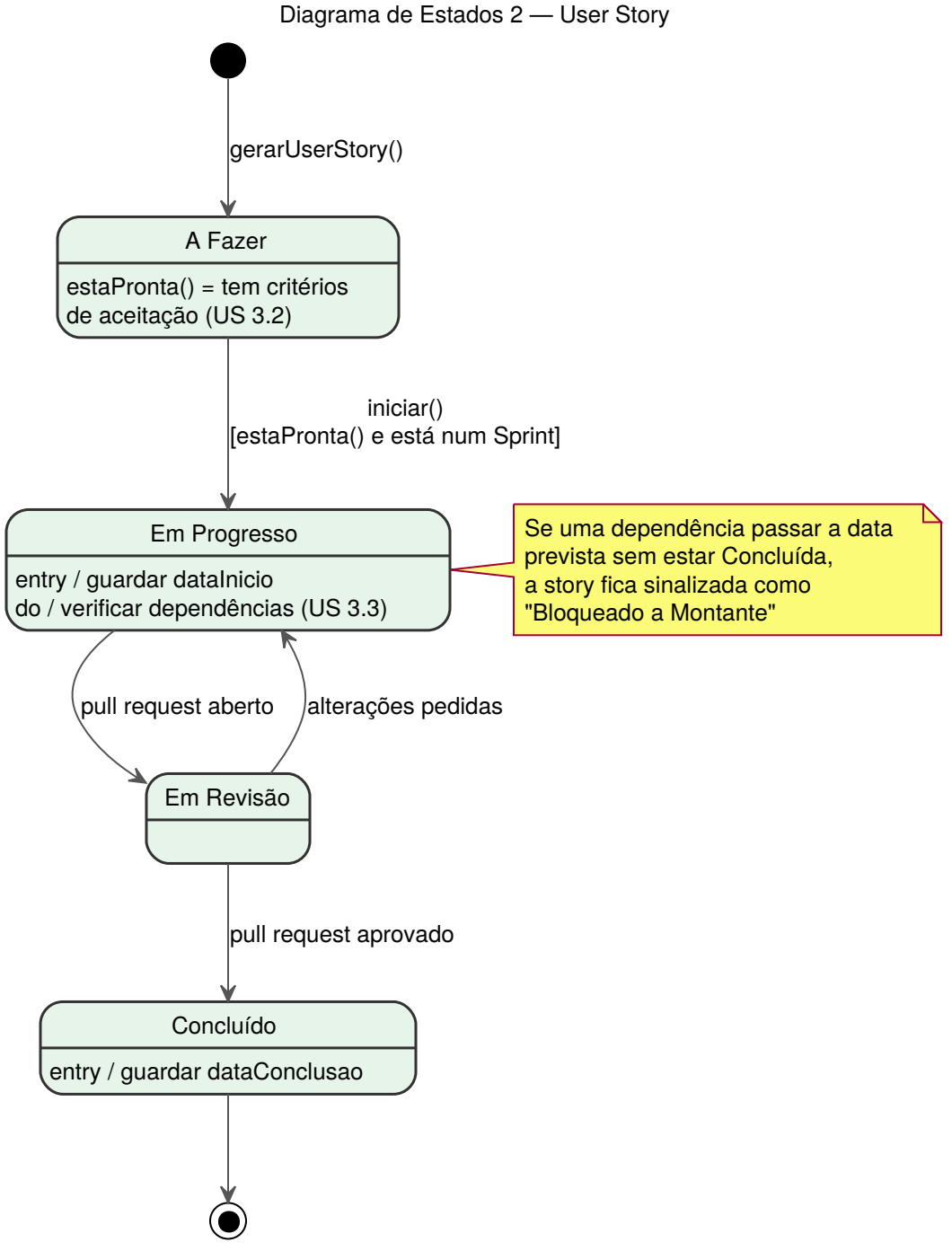
- **Estados iguais em todo o projeto:** os nomes dos estados são os das enumerações `EstadoPedido` e `EstadoUserStory` do diagrama de classes e do Glossário de Estados das Personas.
- **Datas guardadas nas transições:** `dataSubmissao` e `dataLancamento` permitem calcular o Lead Time; `dataInicio` e `dataConclusao` permitem calcular o Cycle Time (US 4.1).
- **Ligação entre os dois ciclos de vida:** o Pedido passa a Em Desenvolvimento por causa de uma User Story, o que torna o progresso visível para o cliente sem lhe mostrar detalhe técnico.
- **Bloqueios não são um estado:** uma story bloqueada continua Em Progresso e é apenas sinalizada (`estaBloqueada()`), porque o trabalho pode continuar noutras partes.

6 Ligação aos outros artefactos

- **Classes (EDS-13):** operações `aprovar()`, `rejeitar()`, `fundirCom()`, `gerarUserStory()`, `iniciar()`, `estaPronta()` e `estaBloqueada()`.
- **Diagramas de sequência 1, 2 e 4:** mostram as mesmas transições do ponto de vista das mensagens entre componentes.

Os diagramas estão nas páginas seguintes, em formato vetorial, e podem ser ampliados sem perder qualidade. O código PlantUML que os gera está nas últimas páginas e pode ser colado em `planttext.com` para o editar.


Diagrama de Estados 1 — Pedido
submeter()
Submetido
entry / guardar dataSubmissao
triagem concluída pelo Worker
(com ou sem sugestões da IA)
Em Triagem
aprovar()
[esforço e impacto definidos]
rejeitar(justificacao)
fundirCom(original) /
transferir votos e anexos
Aprovado
Rejeitado
associado a uma Release /
notificar cliente
Planeado
As transições para Planeado,
Em Desenvolvimento e Lançado
enviam e-mail a quem submeteu
ou votou no pedido (US 2.3)
1.ª User Story passa a EM_PROGRESSO /
notificar cliente
Em Desenvolvimento
Release publicada /
notificar cliente
Lançado
entry / guardar dataLancamento


Diagrama de Estados 2 — User Story
gerarUserStory()
A Fazer
estaPronta() = tem critérios
de aceitação (US 3.2)
iniciar()
[estaPronta() e está num Sprint]
Em Progresso
entry / guardar dataInicio
do / verificar dependências (US 3.3)
Se uma dependência passar a data
prevista sem estar Concluída,
a story fica sinalizada como
"Bloqueado a Montante"
pull request aberto
alterações pedidas
Em Revisão
pull request aprovado
Concluído
entry / guardar dataConclusao

Código PlantUML

est1_pedido.puml

```
@startuml est1_pedido
skinparam shadowing false
skinparam defaultFontName Helvetica
skinparam defaultFontSize 13
skinparam dpi 96
skinparam state {
  BackgroundColor #FFF4E0
  BorderColor #333333
  ArrowColor #555555
}
title Diagrama de Estados 1 -- Pedido

[*] --> Submetido : submeter()
Submetido --> EmTriagem : triagem concluída pelo Worker\n(com ou sem sugestões da IA)
EmTriagem --> Aprovado : aprovar()\n[esforço e impacto definidos]
EmTriagem --> Rejeitado : rejeitar(justificacao)
EmTriagem --> [*] : fundirCom(original) /\ntransferir votos e anexos
Rejeitado --> [*]
Aprovado --> Planeado : associado a uma Release /\nnotificar cliente
Planeado --> EmDesenvolvimento : 1.ª User Story passa a EM_PROGRESSO /\nnotificar cliente
EmDesenvolvimento --> Lancado : Release publicada /\nnotificar cliente
Lancado --> [*]

state "Em Triagem" as EmTriagem
state "Em Desenvolvimento" as EmDesenvolvimento
state "Lançado" as Lancado

Submetido : entry / guardar dataSubmissao
Lancado : entry / guardar dataLancamento

note right of Planeado
  As transições para Planeado,
  Em Desenvolvimento e Lançado
  enviam e-mail a quem submeteu
  ou votou no pedido (US 2.3)
end note
@enduml
```

est2_userstory.puml

```
@startuml est2_userstory
skinparam shadowing false
skinparam defaultFontName Helvetica
skinparam defaultFontSize 13
skinparam dpi 96
skinparam state {
  BackgroundColor #E6F4EA
  BorderColor #333333
  ArrowColor #555555
}
title Diagrama de Estados 2 -- User Story

state "A Fazer" as AFazer
state "Em Progresso" as EmProgresso
state "Em Revisão" as EmRevisao
state "Concluído" as Concluido

```

```
[*] --> AFazer : gerarUserStory()
AFazer --> EmProgresso : iniciar()\n[estaPronta() e está num Sprint]
EmProgresso --> EmRevisao : pull request aberto
EmRevisao --> EmProgresso : alterações pedidas
EmRevisao --> Concluido : pull request aprovado
Concluido --> [*]

AFazer : estaPronta() = tem critérios\nde aceitação (US 3.2)
EmProgresso : entry / guardar dataInicio
EmProgresso : do / verificar dependências (US 3.3)
Concluido : entry / guardar dataConclusao

note right of EmProgresso
  Se uma dependência passar a data
  prevista sem estar Concluída,
  a story fica sinalizada como
  "Bloqueado a Montante"
end note
@enduml
```